

Construyendo un Dashboard Financiero con Flexdashboard

Métricas en tiempo real de una empresa: EDA, KPIs y visualizaciones reactivas

Sebastián Egea Santibañez (segana@fen.uchile.cl)

Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-10-05

Objetivo de la clase

En esta clase construiremos un **dashboard financiero interactivo** usando **Flexdashboard**, una herramienta que combina R Markdown con dise ˜o responsivo tipo Bootstrap.

El dashboard presentar ˆmtricas clave de **TechCorp Inc.**, empresa ficticia de tecnolog ˆa, con ˆan lisis de:

- Ingresos trimestrales y m ˆrgenes
- ˆAn lisis de rentabilidad (ROE, ROA)
- Flujos de efectivo
- Comparaci ˆn contra competencia
- Indicadores de salud financiera

Un dashboard bien construido transforma n ˆmeros en historias visuales que comunican decisiones de inversi ˆn a no-especialistas.

Resultado final

Al terminar tendremos un dashboard con:

- 3 pesta ˆas: Resumen ejecutivo, ˆAn lisis de ingresos, Solidez financiera
- Valueboxes mostrando KPIs principales (Ingresos, Margen neto, ROE actual)
- Gr ˆficos interactivos (plotly) con series de tiempo
- Tablas de comparaci ˆn contra competidores
- Indicadores de tendencia (?/?) en color rojo/verde
- Descargas de datos en CSV

Idea central: un dashboard efectivo jerarquiza informaci ˆn: primero lo importante, luego lo detallado. El usuario debe entender la salud financiera de la empresa en 10 segundos.

Preparaci ˆn

Instala los paquetes una sola vez:

```
install.packages(c("flexdashboard", "plotly", "dplyr", "tidyr", "ggplot2", "scales"))
```

Flexdashboard es un paquete que extiende R Markdown. No ejecutas un archivo .R como Shiny, sino que renderizas un .Rmd o .qmd que genera un HTML interactivo.

Fase 1 Estructura b ˆsica de un dashboard

La anatom ˆa de un dashboard Flexdashboard es simple:

```

---
title: "Mi Dashboard"
output: flexdashboard::flex_dashboard:
  orientation: columns
  vertical_layout: fill
---

```

Caja 1

```

[R code block]
plot(1:10)
[/R code block]

```

Caja 2

Texto o m s c digo.

Qu aprendemos:

- `flexdashboard::flex_dashboard` es el formato de salida
- `orientation: columns` organiza en columnas (alternativamente `rows`)
- `vertical_layout: fill` hace que los gr ficos llenen el espacio disponible
- Cada `###` T tulo crea una caja (card)

Esta estructura se renderiza como un sitio web responsivo autom ticamente.

Fase 2 Agregar pesta as (tabs)

Los dashboards complejos usan pesta as para organizar temas:

```

---
title: "Dashboard Financiero"
output: flexdashboard::flex_dashboard
---

## Resumen Ejecutivo {data-icon="fa-chart-line"}

### Ingresos

[R code block]
# Gr fico de ingresos
[/R code block]

### M rgenes

[R code block]
# Gr fico de m rgenes

```

```
[/R code block]

## Analisis Detallado {data-icon="fa-chart-bar"}

### Comparaci n

[R code block]
# Tabla de competencia
[/R code block]
```

Estructura:

- ## Nombre {data-icon="..."} crea una pesta a
- Las ### Cajas dentro de cada pesta a son tarjetas
- Usa Font Awesome icons para los botones de pesta as

Fase 3 Cargar datos de la empresa

Crearemos datos ficticios de **TechCorp Inc.**, empresa de software fundada en 2018:

```
# Datos hist ricos de TechCorp Inc. (ficticio)
# Trimestres desde Q1 2021 a Q3 2024

datos_techcorp <- data.frame(
  trimestre = c("Q1-2021", "Q2-2021", "Q3-2021", "Q4-2021",
                "Q1-2022", "Q2-2022", "Q3-2022", "Q4-2022",
                "Q1-2023", "Q2-2023", "Q3-2023", "Q4-2023",
                "Q1-2024", "Q2-2024", "Q3-2024"),
  ingresos = c(12, 14, 16, 19, 22, 25, 28, 32,
               37, 41, 46, 52, 58, 65, 72), # millones USD
  costo_operativo = c(8, 9, 10, 12, 13, 15, 16, 18,
                      20, 22, 24, 27, 29, 32, 35),
  ganancia_neta = c(2, 3, 4, 5, 6, 7, 9, 11,
                    13, 15, 17, 20, 22, 25, 28)
)

# Calcula indicadores
datos_techcorp <- datos_techcorp %>%
  mutate(
    margen_neto = round(ganancia_neta / ingresos * 100, 1),
    ganancia_operativa = ingresos - costo_operativo,
    margen_operativo = round(ganancia_operativa / ingresos * 100, 1)
  )

head(datos_techcorp)
```

Este conjunto de datos simula crecimiento consistente de una startup exitosa. Los m rgenes tambi n

mejoran (economía de escala).

Fase 4 Diseñar KPIs con value boxes

Los KPIs (Key Performance Indicators) son números grandes que comunican salud:

```
library(flexdashboard)

# Datos más recientes
ultimo_trimestre <- datos_techcorp %>% slice(n())

# KPI 1: Ingresos actuales
valueBox(
  value = paste0("$", ultimo_trimestre$ingresos, "M"),
  caption = "Ingresos (Q3 2024)",
  icon = "fa-dollar-sign",
  color = "primary"
)

# KPI 2: Margen neto
valueBox(
  value = paste0(ultimo_trimestre$margen_netos, "%"),
  caption = "Margen neto",
  icon = "fa-percentage",
  color = if_else(ultimo_trimestre$margen_netos > 35, "success", "warning")
)

# KPI 3: Ganancia
valueBox(
  value = paste0("$", ultimo_trimestre$ganancia_neta, "M"),
  caption = "Ganancia neta",
  icon = "fa-chart-line",
  color = "success"
)
```

Conceptos:

- `valueBox()` crea una caja grande con un número destacado
 - `color` puede ser `primary`, `success`, `warning`, `danger`
 - Los iconos vienen de Font Awesome (`fa-*`)
-

Fase 5 Graficar series de tiempo

La mayoría del dashboard son gráficos que muestran evolución:

```
library(plotly)

# Gráfico de ingresos trimestrales
plot_ingresos <- datos_techcorp %>%
  plot_ly(x = ~trimestre, y = ~ingresos, type = "scatter", mode = "lines+markers") %>%
  layout(
    title = "Ingresos trimestrales",
    xaxis = list(title = "Trimestre"),
    yaxis = list(title = "Ingresos ($ millones)"),
    hovermode = "x unified"
  )

plot_ingresos
```

Plotly es interactivo: permite zoom, hover, descargar como PNG.

Fase 6 Comparación contra competencia

Para contexto, incluimos competidores ficticios:

```
# Datos de competencia (Q3 2024)
competencia <- data.frame(
  empresa = c("TechCorp", "CloudCo", "DataFlow", "NetworkPlus"),
  ingresos_q3 = c(72, 85, 65, 78),
  margen_netto = c(38.9, 35.2, 32.1, 40.5),
  roe = c(22.5, 18.3, 15.2, 24.1) # Return on Equity%
)

# Visualizar como tabla interactiva
DT::datatable(
  competencia,
  options = list(pageLength = 4),
  rownames = FALSE
)
```

Esta tabla ayuda al lector a ubicar a TechCorp en el mercado.

Fase 7 Calcular indicadores de rentabilidad

ROE (Return on Equity) y ROA (Return on Assets) son claves para inversores:

```
# Supuestos simplificados
activos_totales <- 150 # $ millones
patrimonio <- 90 # $ millones (capital + ganancias retenidas)
```

```

datos_techcorp <- datos_techcorp %>%
  mutate(
    # Ganancia anualizada (4 trimestres)
    ganancia_anual = ganancia_neta * 4,
    # ROA = ganancia anual / activos
    roa = round(ganancia_anual / activos_totales * 100, 1),
    # ROE = ganancia anual / patrimonio
    roe = round(ganancia_anual / patrimonio * 100, 1)
  )

# El ROE de TechCorp es muy fuerte (>30% es excelente)
tail(datos_techcorp[, c("trimestre", "roe", "roa")], 1)

```

Estos n meros indican si la empresa genera valor para accionistas.

Fase 8 Dise o responsivo (mobile-first)

Flexdashboard adapta autom ticamente a pantallas peque as:

```

---
title: "Dashboard Financiero"
output: flexdashboard::flex_dashboard:
  orientation: columns
  vertical_layout: fill
  theme: lumen
---

Column {data-width=500}
-----

### Ingreso Anual

[R code block]
# Gr fico ancho
[/R code block]

Column {data-width=500}
-----

### Margen Neto

[R code block]
# Gr fico angosto
[/R code block]

```

Control:

- `data-width` controla el ancho relativo de columnas
- En móvil, automáticamente se apilan verticalmente
- Estilos (`lumen`, `flatly`, `dark`, `spacelab`) cambian la apariencia

Fase 9 Interactividad con shinyInputs

Flexdashboard puede incluir selectores Shiny para filtrado dinámico:

```
library(shiny)

# En el dashboard:
selectInput(
  inputId = "metrica",
  label = "Selecciona métrica:",
  choices = c("Ingresos" = "ingresos",
              "Margen neto" = "margen_neto",
              "ROE" = "roe")
)

# Luego el gráfico reacciona:
renderPlot({
  datos_techcorp %>%
    plot_ly(x = ~trimestre, y = as.formula(paste0("~", input$metrica))) %>%
    layout(yaxis = list(title = input$metrica))
})
```

Nota: Para esto, el archivo debe tener `runtime: shiny` en el YAML, lo que convierte el dashboard en una app Shiny en lugar de HTML estático.

Fase 10 Reflexión y casos de uso

Cuándo usar Flexdashboard vs Shiny?

Aspecto	Flexdashboard	Shiny
Complejidad	Simple (gráficos estáticos)	Compleja (mucha interactividad)
Deploy	HTML estático (Netlify, GitHub Pages)	Servidor (shinyapps.io, Heroku)
Actualizaciones	Manual (re-render y subir)	En tiempo real (reactivos)
Audiencia	Directivos (reportes)	Analistas (exploración)

Ejemplo de cada:

- **Flexdashboard**: Reporte mensual de finanzas enviado por email
- **Shiny**: Herramienta interactiva para que traders exploren portafolios

Ejercicio:

Crea un dashboard de TechCorp que incluya:

- ☐ 3 pestañas (Resumen, Ingresos, Rentabilidad)
- ☐ 4 value boxes mostrando KPIs
- ☐ 2 gráficos plotly interactivos
- ☐ 1 tabla DT (DataTables) con datos de competencia
- ☐ 1 indicador de tendencia (? Crecimiento del 15 % YoY)
- ☐ Un selector para cambiar entre trimestres

Checklist técnico:

- ☐ Instalaste **flexdashboard**
- ☐ El archivo `.qmd` renderiza sin errores
- ☐ Los value boxes muestran números correctos
- ☐ Los gráficos son interactivos (zoom, hover)
- ☐ El layout es responsivo (prueba en navegador pequeño)
- ☐ Las tablas tienen paginación

Idea clave:

Un dashboard no es solo visualización. Es **comunicación financiera**: - El ejecutivo debe entender en 5 segundos si va bien - El analista debe profundizar sin perder contexto - El inversor debe decidir en 2 minutos

La estructura, los colores y los números deben alinearse con esa intención.

Próximas mejoras

1. Conectar a datos en tiempo real (API de Yahoo Finance, FRED)
2. Agregar pronósticos (modelos ARIMA, Prophet)
3. Implementar alertas automáticas (si ROE cae bajo umbral)
4. Multi-empresa (dropdown para cambiar entre empresas)